**Document Number:** P0653R2
**Date:** 2017-11-09
**Project:** Programming Language C++
**Audience:** Library Working Group
**Author:** Glen Joseph Fernandes (glenjofe@gmail.com)

# Utility to convert a pointer to a raw pointer

This paper proposes adding a new function `to_address` to obtain a raw pointer from an object of a pointer-like type, and an optional customization point in class template `pointer_traits` via a member function of the same name.

## Changes in Revision 2

Prevented use with functions, made the raw pointer overload `constexpr`, and other minor changes to wording suggested by STL during the LWG review.

## Changes in Revision 1

Provided a free function that uses a `pointer_traits` member function only if it exists. This preserves compatibility for any user specializations of `pointer_traits` that do not define a `to_address` member function.

## Motivation

It is often necessary to obtain a raw pointer from an object of any pointer-like type. One common use is writing allocator-aware code where an allocator's `pointer` member type is not a raw pointer type.

Typically the expression `addressof(*p)` is used but this is not well-defined when `p` does not reference storage that has an object constructed in it. This means that using this expression to obtain a raw pointer for the purpose of constructing an object (e.g. via a placement *new-expression* or via an allocator) is incorrect.

A common example of such code:

```
auto p = a.allocate(1);
std::allocator_traits<A>::construct(a, std::addressof(*p), v);
```

The correct code now looks like:

```
auto p = a.allocate(1);
std::allocator_traits<A>::construct(a, std::to_address(p), v);
```

To customize the behavior of this function for a pointer-like type, users can specialize `pointer_traits` for that type and define member function `to_address` accordingly.

### Existing practice

Typically implementors work around this problem by defining a utility like the following:

```
template <class Ptr>
auto to_address(const Ptr& p) noexcept
{
  return to_address(p.operator->());
}

template <class T>
T* to_address(T* p) noexcept
{
  return p;
}
```

This proposal provides a standard library solution, with an optional customization point.

### Why pointer_traits?

The C++ standard library already provides `pointer_traits` for supporting pointer-like types and this `to_address` is the natural inverse of its `pointer_to` member function.

## Implementation

The Boost C++ library collection now contains an implementation of an earlier revision of this proposal in the Core library. That `boost::pointer_traits` implementation is used by several Boost libraries, starting with the 1.65 release.

## Proposed Wording

All changes are relative to N4640.

1. Insert into 20.10.2 [memory.syn] as follows:

```
// 20.10.3, pointer traits
template <class Ptr> struct pointer_traits;
template <class T> struct pointer_traits<T*>;

// 20.10.4, pointer conversion
template <class Ptr> auto to_address(const Ptr& p) noexcept;
template <class T> constexpr T* to_address(T* p) noexcept;
```

2. Insert after 20.10.3 [pointer.traits] as follows:

**20.10.4 Pointer conversion [pointer.conversion]**

`template <class Ptr> auto to_address(const Ptr& p) noexcept;`

*Returns:* `pointer_traits<Ptr>::to_address(p)` if that expression is well-formed, otherwise `to_address(p.operator->())`.

`template <class T> constexpr T* to_address(T* p) noexcept;`

*Requires:* `T` is not a function type. Otherwise the program is ill-formed.
*Returns:* `p`.

3. Add to 20.10.3.1 [pointer.traits.functions] as follows:

`static element_type* to_address(pointer p) noexcept;`

*Remarks:* It is optional for specializations to define this static member function.
*Returns:* A pointer of type `element_type*` that references the same location as the argument `p`.
[Note: This function should be the inverse of `pointer_to`. Specializations can define this function to customize `to_address` [pointer.conversion]. — end note]

## Acknowledgments

Peter Dimov, Jonathan Wakely, and Howard Hinnant helped shape the revised proposal. Peter Dimov suggested the better design as well as reviewed the implementation and tests of the Boost version. Jonathan Wakely pointed out the issue that motivated this proposal. Michael Spencer and Joel Falcou reviewed this paper.

## References

- N4640, Working Draft, Standard for Programming Language C++, http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/n4640.pdf